

tencent / **WeMM-Embedding-4B**   like 23 Follow  Tencent 11.8k

 Feature Extraction
  Transformers
  Safetensors
  sentence-transformers
  Chinese

 English
 qwen3_5
 image-text-to-text
 multimodal-embedding
 text-embedding
 image-embedding

video-embedding
 mrl
 custom_code
  arxiv:2608.24053
  License: apache-2.0


 Deploy 
 → Copy to bucket **NEW**
 Use this model 

 **Model card**
 Files
  xet
  Community 1

WeMM-Embedding-4B

 Hugging Face
  Technical Report

 GitHub WeMM-Embedding

WeMM-Embedding-4B is a universal multimodal embedding model built on Qwen3.5. It accepts text, images, videos, visual documents, and interleaved multimodal inputs, and returns a 2,560-dimensional L2-normalized embedding. Audio input is not supported.

Installation

```
pip install torch transformers==5.2.0 "qwen-
"sentence-transformers>=5.7.0" "accelerate"
```

Transformers

```
import torch
from qwen_vl_utils import process_vision_info
from transformers import AutoModel, AutoProc
```



Safetensors

Model size 5B params

Tensor type BF16  Chat template

 Files info

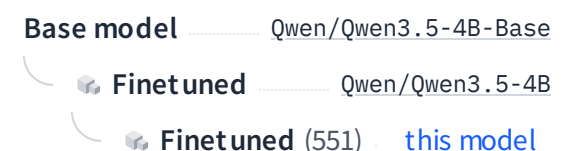
Inference Providers **NEW**

 Feature Extraction

This model isn't deployed by any Inference Provider.

 Ask for provider support

Model tree for tencent/WeMM-Embe...



 Collection including tencent/WeMM...

 **Paper for** tencent/WeMM-Embeddin...

WeMM-Embedding: WeChat Multi-...

 Paper • 2608.24053 • Publi... •  63

```
model_id = "tencent/WeMM-Embedding-4B"
processor = AutoProcessor.from_pretrained(model_id)
model = AutoModel.from_pretrained(
    model_id, trust_remote_code=True, dtype=torch.float16
).cuda().eval()

messages = [{"role": "user", "content": [
    {"type": "image", "image": "/path/to/image.jpg"},
    {"type": "video", "video": "/path/to/video.mp4"},
    {"type": "text", "text": "This can be any text."}
]}]

text = processor.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True
)

images, videos, video_kwargs = process_vision_messages(
    messages,
    image_patch_size=16,
    return_video_kwargs=True,
    return_video_metadata=True,
)

if videos is not None:
    videos, video_metadata = zip(*videos)
    videos, video_metadata = list(videos), list(video_metadata)
else:
    video_metadata = None

inputs = processor(
    text=text,
    images=images,
    videos=videos,
    video_metadata=video_metadata,
    return_tensors="pt",
    **video_kwargs,
).to("cuda")

with torch.inference_mode():
    embedding = model.embedding(**inputs)
```

Use any subset of the content items to encode text, image, or video independently.

Sentence Transformers

```
from sentence_transformers import SentenceTransformer

model_id = "tencent/WeMM-Embedding-4B"
model = SentenceTransformer(model_id, trust_remote_code=True)

queries = [
    "Which Llama 4 model variants are available?",
    "How is mapo tofu prepared?",
]

documents = [
    {
        "image": "https://huggingface.co/datasets/laion-aesthetics-35k-sample-image-text-pairs/resolve/main/00000000000000000000000000000000.jpg",
        "text": "Represent this image.",
    },
    {
        "video": "https://huggingface.co/datasets/laion-aesthetics-35k-sample-image-text-pairs/resolve/main/00000000000000000000000000000000.mp4",
        "text": "Represent this video.",
    },
]

query_embeddings = model.encode_query(queries)
document_embeddings = model.encode_document(documents)

print(query_embeddings.shape, document_embeddings.shape)
# (2, 2560) (3, 2560)

similarities = model.similarity(query_embeddings, document_embeddings)
print(similarities)
# tensor([[ 0.0713,  0.4782, -0.0742],
#         [ 0.7497,  0.0909,  0.4030]])
```

Each input is a string, a URL or path, a PIL.Image, or a dict combining image, video, and text keys. Put image or video before text so the prompt matches the ordering used above. Chat messages such as

```
{"role": "user", "content": [{"type": "image", "image": ...}, {"type": "text", "text": ...}]}
```

are also accepted, which is the way to interleave several images or videos in one input.

Matryoshka Embeddings

```
d = 256
embedding_d = torch.nn.functional.normalize(
```

With Sentence Transformers, pass `truncate_dim` and let it renormalize:

```
embeddings_d = model.encode_document(document
```

Use a dimension listed in `model.config.matryoshka_dimensions`.

Serving

vLLM 0.27.0:

```
MODEL_PATH=/path/to/WeMM-Embedding-4B
vllm serve "$MODEL_PATH" \
  --runner pooling \
  --chat-template "$MODEL_PATH/embedding_chat
```

SGLang 0.5.9:

```
MODEL_PATH=/path/to/WeMM-Embedding-4B
python patch_sglang_video.py
python -m sglang.launch_server \
  --model-path "$MODEL_PATH" \
  --is-embedding \
  --enable-precise-embedding-interpolation
```

Evaluation

MMEB-v2

Results on 78 datasets from Table 1 of the [technical report](#). Image and video tasks use Hit@1, while visual-document tasks use NDCG@5. Higher is better.

Model	Size	AVG	Image	Video	VisDoc
VLM2Vec	2B	47.8	59.7	29.0	44.0
GME	2B	55.4	51.9	33.9	76.8
VLM2Vec-V2	2B	59.3	64.9	34.9	69.2
Qwen3-VL-Embedding	2B	73.2	75.0	61.9	79.2
DME-Small†	2B	74.8	75.9	65.6	79.9
WeMM-Embedding	2B	77.9	79.6	70.8	80.7
WeMM-Embedding	4B	79.2	80.8	72.1	82.0
VLM2Vec	8B	53.2	65.5	34.0	49.1
GME	8B	59.2	56.0	38.6	79.3
Qwen3-VL-Embedding	8B	77.8	80.1	67.1	82.4
DME-Medium†	9B	78.4	79.8	70.8	82.0
WeMM-Embedding	9B	80.6	81.9	74.3	83.3

† Closed-source leaderboard submission without publicly released model weights or a public inference endpoint.

MMEB-v3

Results on all 190 tasks from Table 2 of the [technical report](#). V3-All includes the 78 MMEB-v2 tasks, 53 text tasks, 47 agent tasks, 11 audio tasks, and MCMR. Unsupported tasks are assigned a score of zero.

Model	Size	V3-All	Text	Agent	MCMR	Audio
VLM2Vec-V2	2B	38.3	24.5	28.7	4.1	0.0
Omni-Embed-Nemotron	3B	43.5	39.2	36.5	26.1	36.5
E5-Omni	3B	44.6	26.7	36.9	31.9	30.8
Qwen3-VL-Embedding	2B	50.9	39.2	39.3	42.0	0.0
WeMM-Embedding	2B	56.0	45.3	45.1	42.5	0.0
WeMM-Embedding	4B	58.2	47.9	49.0	41.9	0.0
WAVE	7B	26.3	13.7	11.3	8.9	31.8
VLM2Vec	8B	32.9	22.2	19.7	0.9	0.0
LCO-Embedding-Omni	7B	40.6	32.4	27.8	20.0	43.2
GME	8B	43.6	37.1	35.6	27.3	0.0
E5-Omni	7B	47.1	26.9	36.7	41.1	43.0
Tianmu-Emb-Uni	8B	53.3	43.6	39.4	38.8	38.9

Model	Size	V3- All	Text	Agent	MCMR	Aud
Qwen3-VL- Embedding	8B	53.5	42.5	38.4	38.0	0.0
WeMM- Embedding	9B	59.5	48.8	51.0	49.3	0.0

Text results use NDCG@5; agent, MCMR, and audio results use Hit@1.

Citation

If you find this repository useful, please consider giving a star 🌟 and citation

```
@article{wemm-embedding,  
  title={WeMM-Embedding: WeChat Multi-Modal Embedding},  
  author={Junjie Zhou and Ke Mei and Lei Chen},  
  year={2026},  
  eprint={2608.24053},  
  archivePrefix={arXiv},  
  primaryClass={cs.CV},  
  url={https://arxiv.org/abs/2608.24053}  
}
```

License

WeMM-Embedding-4B, including the code, model parameters, and weights made publicly available by Tencent, is licensed under the [Apache License 2.0](#). Third-party components remain subject to their respective original licenses.